## Randomized Search

Random search is an optimization algorithm that explores the hyperparameter space by randomly sampling hyperparameters from a distribution. The idea is that by exploring a wide range of hyperparameters, the algorithm can identify optimal hyperparameter settings faster than grid search.

For example, consider a model with two hyperparameters: learning rate and regularization strength. Instead of exhaustively searching through a set of combinations, random search would randomly sample values of learning rate and regularization strength from a given distribution. The model would then be trained with these hyperparameters, and the performance would be evaluated. This process would be repeated a specified number of times, and the best-performing hyperparameters would be chosen.

The distribution from which the hyperparameters are sampled can be uniform, normal, or any other distribution that is appropriate for the hyperparameters being optimized. The choice of distribution can greatly affect the performance of the algorithm, so it is important to choose it wisely.

### Advantages of Random Search

- **Efficiency:** Random search is often more efficient than grid search because it explores the hyperparameter space more efficiently. This is because it does not waste time exploring areas of the space that are unlikely to be optimal.
- **Better Performance:** Random search is also less likely to get stuck in local minima than grid search. This is because it explores a wider range of hyperparameters, making it more likely to find the global minimum.
- **Less Sensitive to Noise:** Random search is less sensitive to noisy evaluations of the objective function than other optimization algorithms because it averages over many random samples.

### Disadvantages of Random Search

- **More Computationally Intensive:** Random search can be more computationally intensive than other optimization algorithms because it requires more random samples to achieve the same level of accuracy.
- **No Guarantees:** Random search does not provide any guarantees about the quality of the solution. It only finds the best solution that it discovers through random sampling.
- **Requires Tuning:** Random search requires tuning of the hyperparameters of the search algorithm itself, such as the number of random samples and the distribution from which the hyperparameters are sampled.

### Implementing Random Search

Implementing random search is relatively easy. Here is a simple Python implementation:

```
import numpy as np
from sklearn.model_selection import RandomizedSearchCV
from sklearn.ensemble import RandomForestClassifier
# define hyperparameter space
param_dist = {
    'n_estimators': [10, 100, 1000],
    'max_features': ['sqrt', 'log2'],
    'max_depth': [None, 10],
    'min_samples_split': [2, 10, 100],
    'min_samples_leaf': [1, 10, 100]
}
# define model
model = RandomForestClassifier()
# define search algorithm
search = RandomizedSearchCV(estimator=model, param_distributions=param_dist, n_iter=100)
```

```
# fit search algorithm
search.fit(X_train, y_train)
```

In this example, we define a hyperparameter space consisting of five hyperparameters for a random forest classifier. The search algorithm is defined using the RandomizedSearchCV function from scikit-learn, which takes as input the model, the hyperparameter space, and the number of random samples to draw (n_iter). The search algorithm is then fit to the training data (X_train, y_train), and the best-performing hyperparameters are returned.

## Simulated annealing (SA) Algorithm

Simulated annealing (SA) is a probabilistic algorithm that uses a random search to find the best solution to an optimization problem in artificial intelligence (AI). It's inspired by the process of annealing metal, where a material is refined by heating and cooling.

How it works

1. Start with an initial solution
2. Randomly generate new solutions
3. Accept the new solution if it's better than the current solution
4. Accept the new solution with a certain probability if it's worse than the current solution
5. Gradually reduce the probability of accepting worse solutions
6. Repeat until you reach the lowest minima or hit a stopping criteria

When to use SA

SA is often used when the search space is discrete, such as in: traveling salesman problem, boolean satisfiability problem, protein structure prediction, and job-shop scheduling.

**The benefits of using simulated annealing include:**

1. The ability to find global optima.
2. The ability to escape from local optima.
3. The ability to handle constraints.
4. The ability to handle noisy data.
5. The ability to handle discontinuities.
6. The ability to find solutions in a fraction of the time required by other methods.
7. The ability to find solutions to problems that are difficult or impossible to solve using other methods.

**What are the drawbacks of using simulated annealing?**

Simulated annealing is a technique used in AI to find solutions to optimization problems. It is based on the idea of slowly cooling a material in order to find the lowest energy state, or the most optimal solution.

However, simulated annealing can be slow and may not always find the best solution. Additionally, it can be difficult to tune the parameters of the algorithm, which can lead to sub-optimal results.

Simulated Annealing is an optimization algorithm designed to search for an optimal or near-optimal solution in a large solution space. The name and concept are derived from the process of annealing in metallurgy, where a material is heated and then slowly cooled to remove defects and achieve a stable crystalline structure. In Simulated Annealing, the "heat" corresponds to the degree of randomness in the search process, which decreases over time (cooling schedule) to refine the solution. The method is widely used in combinatorial optimization, where problems often have numerous local optima that standard techniques like gradient descent might get stuck in. Simulated Annealing excels in escaping these local

minima by introducing controlled randomness in its search, allowing for a more thorough exploration of the solution space.

The algorithm starts with an initial solution and a high "temperature," which gradually decreases over time. Here's a step-by-step breakdown of how the algorithm works:

- **Initialization:** Begin with an initial solution So*So* and an initial temperature To*To*. he temperature controls how likely the algorithm is to accept worse solutions as it explores the search space.
- **Neighborhood Search**: At each step, a new solution S′*S′* is generated by making a small change (or perturbation) to the current solution S.
- **Objective Function Evaluation:** The new solution S' is evaluated using the objective function. If S' provides a better solution than S, it is accepted as the new solution.
- **Acceptance Probability**: If S' is worse than S, it may still be accepted with a probability based on the temperature and the difference in objective function values. The acceptance probability is given by:

$$P(accept)=e-\Delta ETP(accept)=e-T\Delta E$$

- Cooling Schedule: After each iteration, the temperature is decreased according to a predefined cooling schedule, which determines how quickly the algorithm converges. Common cooling schedules include linear, exponential, or logarithmic cooling.
- Termination: The algorithm continues until the system reaches a low temperature (i.e., no more significant improvements are found), or a predetermined number of iterations is reached.

**Cooling Schedule and Its Importance**

The cooling schedule plays a crucial role in the performance of Simulated Annealing. If the temperature decreases too quickly, the algorithm might converge prematurely to a suboptimal solution (local optimum). On the other hand, if the cooling is too slow, the algorithm may take an excessively long time to find the optimal solution. Hence, finding the right balance between exploration (high temperature) and exploitation (low temperature) is essential.

**Advantages of Simulated Annealing**

- **Ability to Escape Local Minima**: One of the most significant advantages of Simulated Annealing is its ability to escape local minima. The probabilistic acceptance of worse solutions allows the algorithm to explore a broader solution space.
- **Simple Implementation**: The algorithm is relatively easy to implement and can be adapted to a wide range of optimization problems.
- **Global Optimization**: Simulated Annealing can approach a global optimum, especially when paired with a well-designed cooling schedule.
- **Flexibility:** The algorithm is flexible and can be applied to both continuous and discrete optimization problems.

**Limitations of Simulated Annealing**

- **Parameter Sensitivity:** The performance of Simulated Annealing is highly dependent on the choice of parameters, particularly the initial temperature and cooling schedule.
- **Computational Time:** Since Simulated Annealing requires many iterations, it can be computationally expensive, especially for large problems.
- **Slow Convergence:** The convergence rate is generally slower than more deterministic methods like gradient-based optimization.

| Parameters | Hill Climbing | Simulated Annealing |
|---|---|---|
| Introduction | Hill Climbing is a heuristic optimization process that iteratively advances towards a better solution at each step in order to find the best solution in a given search space. | Simulated Annealing is a probabilistic optimization algorithm that simulates the metallurgical annealing process in order to discover the best solution in a given search area by accepting less-than-ideal solutions with a predetermined probability. |
| Objective | By iteratively progressing towards a better solution at each stage, Hill Climbing seeks to locate the ideal solution within a predetermined search space. | Simulated annealing seeks the global optimum in a given search space by accepting poorer answers with a predetermined probability. This allows it to bypass local optimum conditions. |
| Strategy | In order to iteratively move towards the best answer at each stage, Hill Climbing employs a greedy method. It only accepts solutions that are superior to the ones already in place. | Simulated annealing explores the search space and avoids local optimum by employing a probabilistic method to accept a worse solution with a given probability. As the algorithm advances, the likelihood of accepting an inferior answer diminishes. |
| Local vs. Global Optima | Hill Climbing comes to an end after a certain number of iterations or when it achieves a local optimum. | Simulated annealing is more efficient at locating the global optimum than Hill Climbing, particularly for complicated situations with numerous local optima. Simulated annealing is slower than Hill Climbing. |
| Tuning Parameters | Hill Climbing has no tuning parameters. | The beginning temperature, cooling schedule, and acceptance probability function are only a few of the tuning factors for Simulated Annealing. |
| Applications | Many different applications, including image processing, machine learning, and gaming, use hill climbing. | Several fields, including logistics, scheduling, and circuit design, use simulated annealing. |

**Comparison to Other Optimization Techniques**

Simulated Annealing is often compared to other global optimization techniques like Genetic Algorithms (GA) and Particle Swarm Optimization (PSO).

- Simulated Annealing vs. Genetic Algorithms: While both methods are probabilistic and capable of escaping local minima, GAs use a population of solutions and evolve them over generations, whereas SA works with a single solution that is iteratively improved.
- Simulated Annealing vs. Gradient Descent: Gradient descent is faster but can easily get stuck in local minima. Simulated Annealing, on the other hand, can escape local minima but is generally slower.

**Applications of Simulated Annealing**

- Simulated Annealing has found widespread use in various fields due to its versatility and effectiveness in solving complex optimization problems. Some notable applications include:
- Traveling Salesman Problem (TSP): In combinatorial optimization, SA is often used to find near-optimal solutions for the TSP, where a salesman must visit a set of cities and return to the origin, minimizing the total travel distance.
- VLSI Design: SA is used in the physical design of integrated circuits, optimizing the layout of components on a chip to minimize area and delay.
- Machine Learning: In machine learning, SA can be used for hyperparameter tuning, where the search space for hyperparameters is large and non-convex.
- Scheduling Problems: SA has been applied to job scheduling, minimizing delays and optimizing resource allocation.
- Protein Folding: In computational biology, SA has been used to predict protein folding by optimizing the conformation of molecules to achieve the lowest energy state.

**Genetic Algorithms**

Genetic Algorithms(GAs) are adaptive heuristic search algorithms that belong to the larger part of evolutionary algorithms. Genetic algorithms are based on the ideas of natural selection and genetics. These are intelligent exploitation of random searches provided with historical data to direct the search into the region of better performance in solution space. **They are commonly used to generate high-quality solutions for optimization problems and search problems.**

Genetic algorithms (GAs) are a search heuristic used in artificial intelligence (AI) to solve problems. They are based on natural selection and are often used to optimize solutions.

How do GAs work?

1. Start with a population of candidate solutions
2. Evaluate the solutions
3. Repeat the following steps for multiple generations
    - **Mutation**: Introduce random changes to the genetic information of some individuals
    - **Crossover**: Swap genetic material between individuals
    - **Selection**: Only allow the fittest individuals to survive
4. Repeat until a termination condition is met

- **Applications**

GAs are used in many areas of AI, including machine learning, scheduling, routing, and parameter tuning. They are particularly useful for problems with large search spaces, or when traditional optimization methods are not effective.

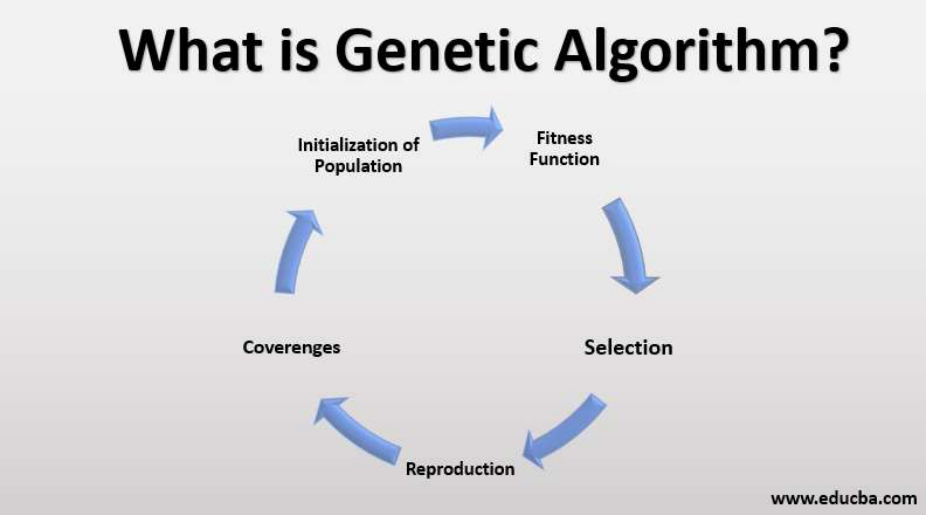
- **Related techniques**

GAs are part of a collection of techniques called evolutionary algorithms, which are inspired by the process of biological evolution.

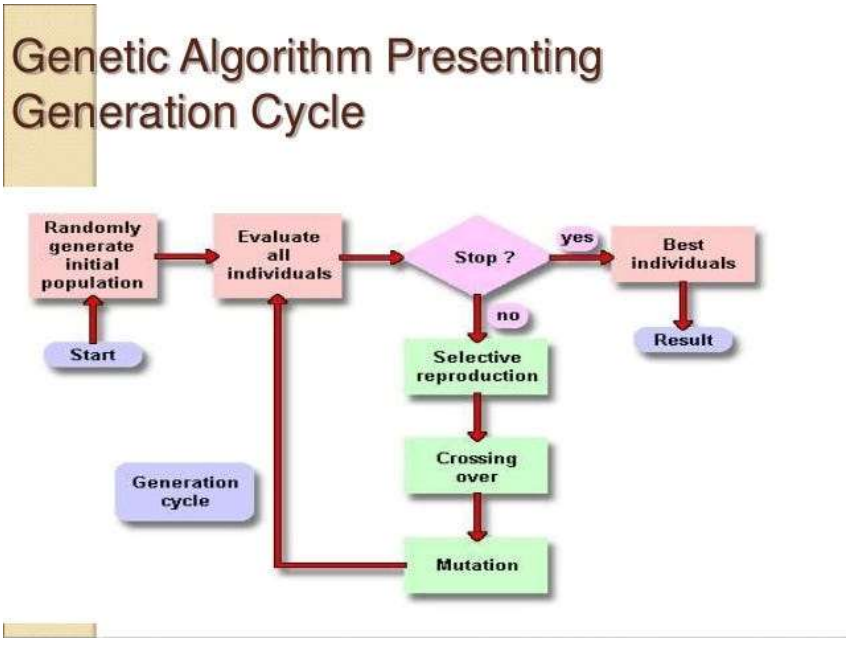
- **Other names**

GAs are also known as biologically-inspired optimization algorithms.

**Genetic algorithms are defined** as a type of computational optimization technique inspired by the principles of natural selection and genetics. They are used to solve complex problems by mimicking the process of evolution to improve a population of potential solutions iteratively




**Genetic algorithms simulate the process of natural selection** which means those species that can adapt to changes in their environment can survive and reproduce and go to the next generation. In simple words, they simulate "survival of the fittest" among individuals of consecutive generations to solve a problem. **Each generation consists of a population of individuals** and each individual represents a point in search space and possible solution. Each individual is represented as a string of character/integer/float/bits. This string is analogous to the Chromosome.




**Foundation of Genetic Algorithms**

Genetic algorithms are based on an analogy with the genetic structure and behavior of chromosomes of the population. Following is the foundation of GAs based on this analogy –

1. Individuals in the population compete for resources and mate
2. Those individuals who are successful (fittest) then mate to create more offspring than others
3. Genes from the "fittest" parent propagate throughout the generation, that is sometimes parents create offspring which is better than either parent.
4. Thus each successive generation is more suited for their environment.

**Search space**

The population of individuals are maintained within search space. Each individual represents a solution in search space for given problem. Each individual is coded as a finite length vector (analogous to chromosome) of components. These variable components are analogous to Genes. Thus a chromosome (individual) is composed of several genes (variable components).


**Fitness Score**

A Fitness Score is given to each individual which **shows the ability of an individual to "compete"**. The individual having optimal fitness score (or near optimal) are sought.

The GAs maintains the population of n individuals (chromosome/solutions) along with their fitness scores.The individuals having better fitness scores are given more chance to reproduce than others. The individuals with better fitness scores are selected who mate and produce **better offspring** by combining chromosomes of parents. The population size is static so the room has to be created for new arrivals. So, some individuals die and get replaced by new arrivals eventually creating new generation when all the mating opportunity of the old population is exhausted. It is hoped that over successive generations better solutions will arrive while least fit die.

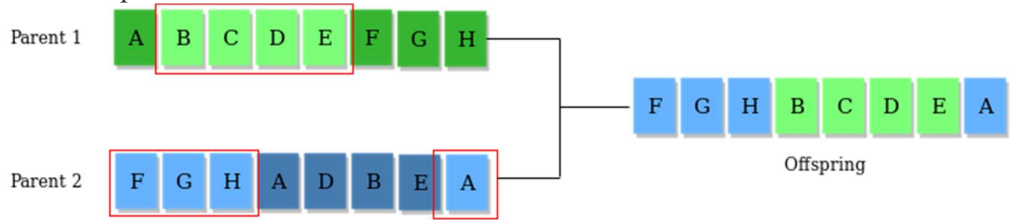
Each new generation has on average more "better genes" than the individual (solution) of previous generations. Thus each new generations have better **"partial solutions"** than previous generations. Once the offspring produced having no significant difference from offspring produced by previous populations, the population is converged. The algorithm is said to be converged to a set of solutions for the problem.

**Operators of Genetic Algorithms**

Once the initial generation is created, the algorithm evolves the generation using following operators –

**1) Selection Operator:** The idea is to give preference to the individuals with good fitness scores and allow them to pass their genes to successive generations.

**2) Crossover Operator:** This represents mating between individuals. Two individuals are selected using selection operator and crossover sites are chosen randomly. Then the genes at these crossover sites are exchanged thus creating a completely new individual (offspring). For example –



**3) Mutation Operator:** The key idea is to insert random genes in offspring to maintain the diversity in the population to avoid premature convergence. For example –


The whole algorithm can be summarized as –

1) Randomly initialize populations p
2) Determine fitness of population
3) Until convergence repeat:
   a) Select parents from population
   b) Crossover and generate new population
   c) Perform mutation on new population
   d) Calculate fitness for new population

**Example problem and solution using Genetic Algorithms**

Given a target string, the goal is to produce target string starting from a random string of the same length. In the following implementation, following analogies are made –

- Characters A-Z, a-z, 0-9, and other special symbols are considered as genes
- A string generated by these characters is considered as chromosome/solution/Individual

**Fitness score** is the number of characters which differ from characters in target string at a particular index. So individual having lower fitness value is given more preference.

**Why use Genetic Algorithms**

- They are Robust
- Provide optimisation over large space state.
- Unlike traditional AI, they do not break on slight change in input or presence of noise

**Application of Genetic Algorithms**

Genetic algorithms have many applications, some of them are –

- Recurrent Neural Network
- Mutation testing
- Code breaking
- Filtering and signal processing
- Learning fuzzy rule base etc

**Traveling Salesman Problem using Genetic Algorithm**

Genetic algorithms are heuristic search algorithms inspired by the process that supports the evolution of life. The algorithm is designed to replicate the natural selection process to carry generation, i.e. survival of the fittest of beings. Standard genetic algorithms are divided into five phases which are:

1. Creating initial population.
2. Calculating fitness.
3. Selecting the best genes.
4. Crossing over.
5. Mutating to introduce variations.

These algorithms can be implemented to find a solution to the optimization problems of various types. One such problem is the Traveling Salesman Problem. The problem says that a salesman is given a set of cities, he has to find the shortest route to as to visit each city exactly once and return to the starting city.
**Approach:** In the following implementation, cities are taken as genes, string generated using these characters is called a chromosome, while a fitness score which is equal to the path length of all the cities mentioned, is used to target a population. Fitness Score is defined as the length of the path described by the gene. Lesser the path length fitter is the gene. The fittest of all the genes in the gene pool survive the population test and move to the next iteration. The number of iterations depends upon the value of a cooling variable. The value of the cooling variable keeps on decreasing with each iteration and reaches a threshold after a certain number of iterations.

**Algorithm:**

1. Initialize the population randomly.
2. Determine the fitness of the chromosome.
3. Until done repeat:
   1. Select parents.
   2. Perform crossover and mutation.
   3. Calculate the fitness of the new population.
   4. Append it to the gene pool.

**Pseudo-code**

```
Initialize procedure GA{
   Set cooling parameter = 0;
   Evaluate population P(t);
   While( Not Done ){
      Parents(t) = Select_Parents(P(t));
      Offspring(t) = Procreate(P(t));
      p(t+1) = Select_Survivors(P(t), Offspring(t));
      t = t + 1;
   }
}
```

**How the mutation works?**
Suppose there are 5 cities: 0, 1, 2, 3, 4. The salesman is in city 0 and he has to find the shortest route to travel through all the cities back to the city 0. A chromosome representing the path chosen can be represented as:

| 0 | 1 | 4 | 2 | 3 | 0 |
|---|---|---|---|---|---|

This chromosome undergoes mutation. During mutation, the position of two cities in the chromosome is swapped to form a new configuration, except the first and the last cell, as they represent the start and endpoint.


Original chromosome had a path length equal to **INT_MAX**, according to the input defined below, since the path between city 1 and city 4 didn't exist. After mutation, the new child formed has a path length equal to **21**, which is a much-optimized answer than the original assumption. This is how the genetic algorithm optimizes solutions to hard problems.

**Time complexity:** O(n^2) as it uses nested loops to calculate the fitness value of each gnome in the population.

**Auxiliary Space**: O(n)

**How the genetic algorithm works**

Let's consider an example that involves optimizing a common task: finding the best route to commute from home to work.

Imagine you want to optimize your daily commute and find the shortest route to go from your home to your workplace. You have multiple possible routes to choose from, each with different distances, traffic conditions, and travel times. You can use a GA to help you find the optimal route.

1**. Encoding the solutions**

In this case, potential solutions can be encoded as permutations of the cities or locations along the commute route. For example, you can represent each possible route as a string of city identifiers, such as "A-B-C-D-E-F," where each letter represents a location (e.g., a street, intersection, or landmark).

**2. Initialization**

Start by creating an initial population of potential routes. You can randomly generate a set of routes or use existing routes as a starting point.

**3. Evaluation**

Evaluate each route in the population by considering factors such as distance, traffic conditions, travel time, and other relevant criteria. The evaluation function should quantify the quality of each route, where lower values indicate better solutions (e.g., shorter distance, less time spent in traffic).

**4. Selection**

Perform a selection process to choose which routes will be part of the next generation. Selection methods aim to favor fitter individuals, in this case, routes with lower evaluation values. Common selection techniques include tournament selection, roulette wheel selection, or rank-based selection.

**5. Crossover**

Apply crossover to create new routes by combining genetic material from two parent routes. For instance, you can select two parent routes and exchange segments of the routes to create two new offspring routes.

**6. Mutation**

Introduce random changes in the routes through mutation. This helps explore new possibilities and avoid getting stuck in local optima. A mutation operation could involve randomly swapping two cities in a route, inserting a new city, or randomly changing the order of a few cities.

**7. New generation**
The offspring generated through crossover and mutation and a few fittest individuals from the previous generation form the new population for the next iteration. This ensures that good solutions are preserved and carried forward.
**8. Termination**
The GA continues the selection, crossover, and mutation process for a fixed number of generations or until a termination criterion is met. Termination criteria can be a maximum number of iterations or reaching a satisfactory solution (e.g., a route with a predefined low evaluation value).
**9. Final solution**
Once the GA terminates, the best solution, typically the route with the lowest evaluation value, represents the optimal or near-optimal route for your daily commute.
By iteratively applying selection, crossover, and mutation, GAs help explore and evolve the population of routes, gradually converging toward the shortest and most efficient route for your daily commute.
It's important to note that GAs require appropriate parameter settings, such as population size, selection strategy, crossover and mutation rates, and termination criteria, to balance exploration and exploitation.

## Examples of Genetic Algorithms

Companies across various industries have used genetic algorithms to tackle a range of challenges. Here are a few recent noteworthy examples of GA:

**1. Google's DeepMind**
DeepMind, a subsidiary of Google, has utilized genetic algorithms in its research on artificial intelligence. One notable example is the AlphaFold project, where DeepMind used GAs to develop a groundbreaking protein-folding algorithm. The algorithm accurately predicted the 3D structures of proteins, which is crucial for understanding their functions and has implications in drug discovery and disease research.
2. Tesla's self-driving tasks
Tesla, the electric vehicle and clean energy company, has implemented genetic algorithms in their autonomous driving technology. The algorithms optimize and fine-tune the neural networks responsible for self-driving tasks. By applying GAs, Tesla can evolve and improve the performance of their autonomous driving systems, enhancing safety and efficiency.
**3. Amazon's logistics operations**
Amazon has leveraged genetic algorithms to optimize its order fulfillment and logistics operations. GAs are used to solve complex routing and scheduling problems, helping Amazon streamline its supply chain and improve delivery efficiency. By evolving and adapting algorithms based on real-time data, Amazon can dynamically optimize its operations to meet customer demands effectively.
**4. Autodesk's design optimization**
Autodesk, a software company specializing in computer-aided design (CAD) and engineering solutions, has incorporated genetic algorithms in its software products. They enable users to apply GAs for optimization, such as finding optimal shapes for mechanical components or generating efficient 3D structures.
**5. Uber's evolutionary optimizer**
Uber, the ride-hailing company, developed an optimization framework called the Evolutionary Optimizer. This project employed genetic algorithms to improve the efficiency of Uber's dynamic pricing system. By evolving and selecting pricing strategies based on historical data and real-time demand patterns, the optimizer maximized the company's revenue while ensuring a fair pricing experience for customers.

**6. Boeing's wing design optimization**
Boeing utilized genetic algorithms for wing design optimization. In projects like the blended wing body (BWB) and the transonic truss-braced wing (TTBW), genetic algorithms were employed to explore various wing shapes, sizes, and configurations. This approach helped Boeing improve aerodynamic efficiency, reduce weight, and enhance fuel efficiency in their aircraft designs.
**7. Ford's vehicle routing optimization**
Ford Motor Company used genetic algorithms for vehicle routing optimization. In their project, Ford employed GAs to determine the optimal routes for their delivery vehicles, considering factors such as traffic conditions, package sizes, and delivery deadlines. This optimization effort helped Ford streamline logistics operations, reduce delivery times, and improve overall efficiency.
**8. Siemens' manufacturing process optimization**
Siemens, a global technology conglomerate, applied genetic algorithms for manufacturing process optimization. In its project, Siemens used GAs to optimize production schedules, machine configurations, and workflow layouts in manufacturing facilities. This approach allowed Siemens to improve production efficiency, reduce downtime, and minimize costs in their manufacturing processes.
**9. NVIDIA's GPU architecture optimization**
NVIDIA utilized genetic algorithms for GPU architecture optimization. GAs were employed to explore and fine-tune the design parameters of graphics processing units, enhancing performance and energy efficiency in AI and gaming applications.
**10. Toyota's supply chain optimization**
Toyota applied genetic algorithms to optimize its global supply chain. GAs were used to optimize production schedules, logistics routes, and inventory management, improving overall supply chain efficiency and reducing costs.
These examples showcase how companies across different sectors leverage genetic algorithms to solve complex optimization problems, improve efficiency, and drive innovation in their respective industries.

## What is a neural network?

A neural network is a method in artificial intelligence (AI) that teaches computers to process data in a way that is inspired by the human brain. It is a type of machine learning (ML) process, called deep learning, that uses interconnected nodes or neurons in a layered structure that resembles the human brain. It creates an adaptive system that computers use to learn from their mistakes and improve continuously. Thus, artificial neural networks attempt to solve complicated problems, like summarizing documents or recognizing faces, with greater accuracy.

## Why are neural networks important?

Neural networks can help computers make intelligent decisions with limited human assistance. This is because they can learn and model the relationships between input and output data that are nonlinear and complex. For instance, they can do the following tasks.

**Make generalizations and inferences**
Neural networks can comprehend unstructured data and make general observations without explicit training. For instance, they can recognize that two different input sentences have a similar meaning:

- Can you tell me how to make the payment?
- How do I transfer money?

A neural network would know that both sentences mean the same thing. Or it would be able to broadly recognize that Baxter Road is a place, but Baxter Smith is a person's name.

## What are neural networks used for?

Neural networks have several use cases across many industries, such as the following:

- Medical diagnosis by medical image classification
- Targeted marketing by social network filtering and behavioral data analysis
- Financial predictions by processing historical data of financial instruments
- Electrical load and energy demand forecasting
- Process and quality control
- Chemical compound identification

We give four of the important applications of neural networks below.

**Computer vision**

Computer vision is the ability of computers to extract information and insights from images and videos. With neural networks, computers can distinguish and recognize images similar to humans. Computer vision has several applications, such as the following:

- Visual recognition in self-driving cars so they can recognize road signs and other road users
- Content moderation to automatically remove unsafe or inappropriate content from image and video archives
- Facial recognition to identify faces and recognize attributes like open eyes, glasses, and facial hair
- Image labeling to identify brand logos, clothing, safety gear, and other image details

**Speech recognition**

Neural networks can analyze human speech despite varying speech patterns, pitch, tone, language, and accent. Virtual assistants like Amazon Alexa and automatic transcription software use speech recognition to do tasks like these:

- Assist call center agents and automatically classify calls
- Convert clinical conversations into documentation in real time
- Accurately subtitle videos and meeting recordings for wider content reach

**Natural language processing**

Natural language processing (NLP) is the ability to process natural, human-created text. Neural networks help computers gather insights and meaning from text data and documents. NLP has several use cases, including in these functions:

- Automated virtual agents and chatbots
- Automatic organization and classification of written data
- Business intelligence analysis of long-form documents like emails and forms
- Indexing of key phrases that indicate sentiment, like positive and negative comments on social media
- Document summarization and article generation for a given topic

**Recommendation engines**

Neural networks can track user activity to develop personalized recommendations. They can also analyze all user behavior and discover new products or services that interest a specific user. For example, Curalate, a Philadelphia-based startup, helps brands convert social media posts into sales. Brands use Curalate's intelligent product tagging (IPT) service to automate the collection and curation of user-generated social content. IPT uses neural networks to

automatically find and recommend products relevant to the user's social media activity. Consumers don't have to hunt through online catalogs to find a specific product from a social media image. Instead, they can use Curalate's auto product tagging to purchase the product with ease.

## How do neural networks work?

The human brain is the inspiration behind neural network architecture. Human brain cells, called neurons, form a complex, highly interconnected network and send electrical signals to each other to help humans process information. Similarly, an artificial neural network is made of artificial neurons that work together to solve a problem. Artificial neurons are software modules, called nodes, and artificial neural networks are software programs or algorithms that, at their core, use computing systems to solve mathematical calculations.

**Simple neural network architecture**

A basic neural network has interconnected artificial neurons in three layers:

**Input Layer**

Information from the outside world enters the artificial neural network from the input layer. Input nodes process the data, analyze or categorize it, and pass it on to the next layer.

**Hidden Layer**

Hidden layers take their input from the input layer or other hidden layers. Artificial neural networks can have a large number of hidden layers. Each hidden layer analyzes the output from the previous layer, processes it further, and passes it on to the next layer.

**Output Layer**

The output layer gives the final result of all the data processing by the artificial neural network. It can have single or multiple nodes. For instance, if we have a binary (yes/no) classification problem, the output layer will have one output node, which will give the result as 1 or 0. However, if we have a multi-class classification problem, the output layer might consist of more than one output node.

**Deep neural network architecture**

Deep neural networks, or deep learning networks, have several hidden layers with millions of artificial neurons linked together. A number, called weight, represents the connections between one node and another. The weight is a positive number if one node excites another, or negative if one node suppresses the other. Nodes with higher weight values have more influence on the other nodes.

Theoretically, deep neural networks can map any input type to any output type. However, they also need much more training as compared to other machine learning methods. They need millions of examples of training data rather than perhaps the hundreds or thousands that a simpler network might need.

## What are the types of neural networks?

Artificial neural networks can be categorized by how the data flows from the input node to the output node. Below are some examples:

**Feedforward neural networks**

Feedforward neural networks process data in one direction, from the input node to the output node. Every node in one layer is connected to every node in the next layer. A feedforward network uses a feedback process to improve predictions over time.

**Backpropagation algorithm**

Artificial neural networks learn continuously by using corrective feedback loops to improve their predictive analytics. In simple terms, you can think of the data flowing from the input node to the output node through many different paths in the neural network. Only one path is the correct one that maps the input node to the correct output node. To find this path, the neural network uses a feedback loop, which works as follows:

1. Each node makes a guess about the next node in the path.
2. It checks if the guess was correct. Nodes assign higher weight values to paths that lead to more correct guesses and lower weight values to node paths that lead to incorrect guesses.
3. For the next data point, the nodes make a new prediction using the higher weight paths and then repeat Step 1.

**Convolutional neural networks**

The hidden layers in convolutional neural networks perform specific mathematical functions, like summarizing or filtering, called convolutions. They are very useful for image classification because they can extract relevant features from images that are useful for image recognition and classification. The new form is easier to process without losing features that are critical for making a good prediction. Each hidden layer extracts and processes different image features, like edges, color, and depth.

How to train neural networks?

Neural network training is the process of teaching a neural network to perform a task. Neural networks learn by initially processing several large sets of labeled or unlabeled data. By using these examples, they can then process unknown inputs more accurately.

**Supervised learning**

In supervised learning, data scientists give artificial neural networks labeled datasets that provide the right answer in advance. For example, a deep learning network training in facial recognition initially processes hundreds of thousands of images of human faces, with various terms related to ethnic origin, country, or emotion describing each image.

The neural network slowly builds knowledge from these datasets, which provide the right answer in advance. After the network has been trained, it starts making guesses about the ethnic origin or emotion of a new image of a human face that it has never processed before.

What is deep learning in the context of neural networks?

Artificial intelligence is the field of computer science that researches methods of giving machines the ability to perform tasks that require human intelligence. Machine learning is an artificial intelligence technique that gives computers access to very large datasets and teaches them to learn from this data. Machine learning software finds patterns in existing data and applies those patterns to new data to make intelligent decisions. Deep learning is a subset of machine learning that uses deep learning networks to process data.

**Machine learning vs. deep learning**

Traditional machine learning methods require human input for the machine learning software to work sufficiently well. A data scientist manually determines the set of relevant features that the software must analyze. This limits the software's ability, which makes it tedious to create and manage.

On the other hand, in deep learning, the data scientist gives only raw data to the software. The deep learning network derives the features by itself and learns more independently. It can

analyze unstructured datasets like text documents, identify which data attributes to prioritize, and solve more complex problems.

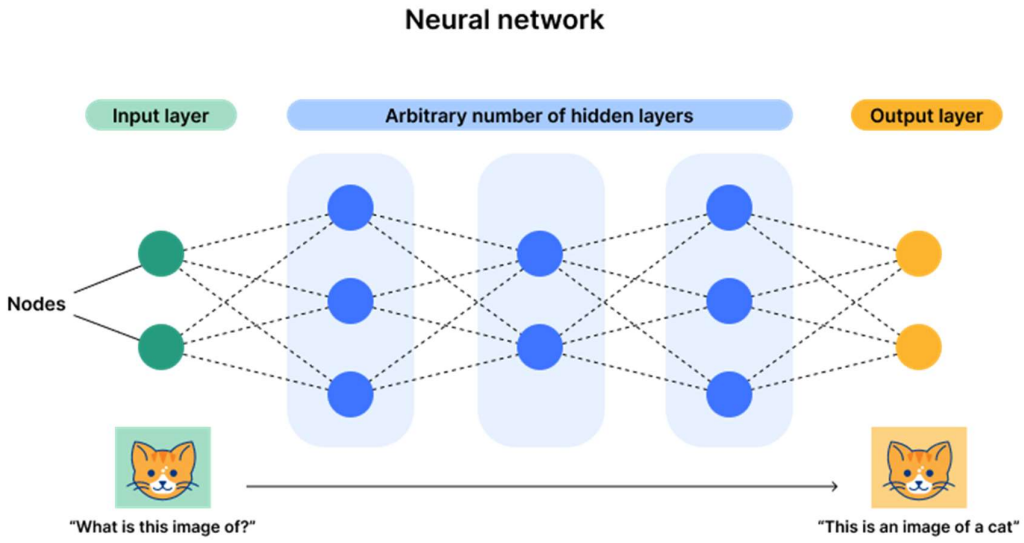
For example, if you were training a machine learning software to identify an image of a pet correctly, you would need to take these steps:

- Find and label thousands of pet images, like cats, dogs, horses, hamsters, parrots, and so on, manually.
- Tell the machine learning software what features to look for so it can identify the image using elimination. For instance, it might count the number of legs, then check for eye shape, ear shape, tail, fur, and so on.
- Manually assess and change the labeled datasets to improve the software's accuracy. For example, if your training set has too many pictures of black cats, the software will correctly identify a black cat but not a white one.
- In deep learning, however, the neural networks would process all the images and automatically determine that they need to analyze the number of legs and the face shape first, then look at the tails last to correctly identify the animal in the image.

**What is a neural network?**

A neural network, or artificial neural network, is a type of computing architecture that is based on a model of how a human brain functions — hence the name "neural." Neural networks are made up of a collection of processing units called "nodes." These nodes pass data to each other, just like how in a brain, neurons pass electrical impulses to each other.

Neural networks are used in machine learning, which refers to a category of computer programs that learn without definite instructions. Specifically, neural networks are used in deep learning — an advanced type of machine learning that can draw conclusions from unlabeled data without human intervention. For instance, a deep learning model built on a neural network and fed sufficient training data could be able to identify items in a photo it has never seen before.




Neural networks make many types of artificial intelligence (AI) possible. Large language models (LLMs) such as ChatGPT, AI image generators like DALL-E, and predictive AI models all rely to some extent on neural networks.

**How do neural networks work?**

Neural networks are composed of a collection of nodes. The nodes are spread out across at least three layers. The three layers are:

- An input layer
- A "hidden" layer
- An output layer

These three layers are the minimum. Neural networks can have more than one hidden layer, in addition to the input layer and output layer.

No matter which layer it is part of, each node performs some sort of processing task or function on whatever input it receives from the previous node (or from the input layer). Essentially, each node contains a mathematical formula, with each variable within the formula weighted differently. If the output of applying that mathematical formula to the input exceeds a certain threshold, the node passes data to the next layer in the neural network. If the output is below the threshold, no data is passed to the next layer.

Imagine that the Acme Corporation has an accounting department with a strict hierarchy. Acme accounting department employees at the manager level approve expenses below $1,000, directors approve expenses below $10,000, and the CFO approves any expenses that exceed $10,000. When employees from other departments of Acme Corp. submit their expenses, they first go to the accounting managers. Any expense over $1,000 gets passed to a director, while expenses below $1,000 stay at the managerial level — and so on.

The accounting department of the Acme Corp. functions somewhat like a neural network. When employees submit their expense reports, this is like a neural network's input layer. Each manager and director is like a node within the neural network.

And, just as one accounting manager may ask another manager for assistance in interpreting an expense report before passing it along to an accounting director, neural networks can be architected in a variety of ways. Nodes can communicate in multiple directions.

**What are the types of neural networks?**

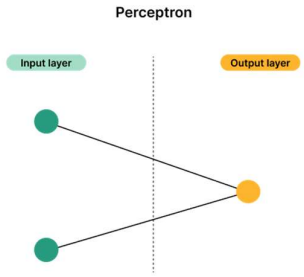
There is no limit on how many nodes and layers a neural network can have, and these nodes can interact in almost any way. Because of this, the list of types of neural networks is ever-expanding. But, they can roughly be sorted into these categories:

- **Shallow neural networks** usually have only one hidden layer
- **Deep neural networks** have multiple hidden layers

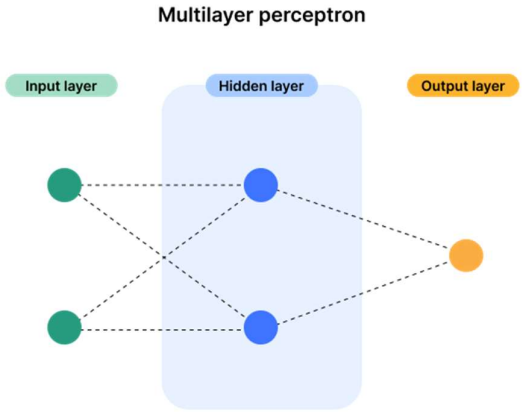
Shallow neural networks are fast and require less processing power than deep neural networks, but they cannot perform as many complex tasks as deep neural networks.

Below is an incomplete list of the types of neural networks that may be used today:

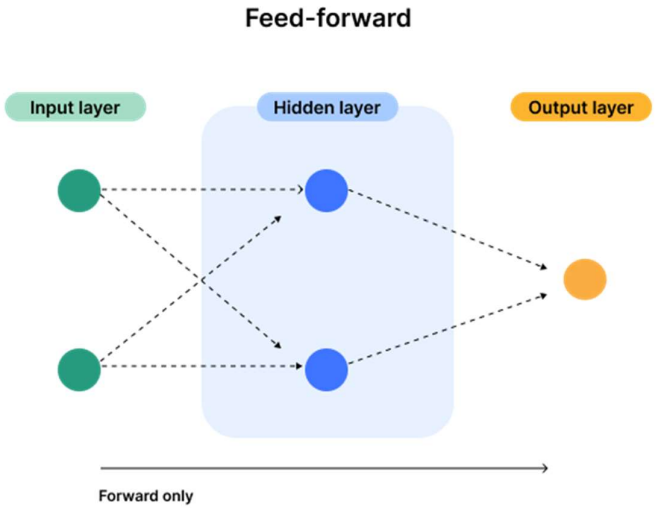
**Perceptron** neural networks are simple, shallow networks with an input layer and an output layer.



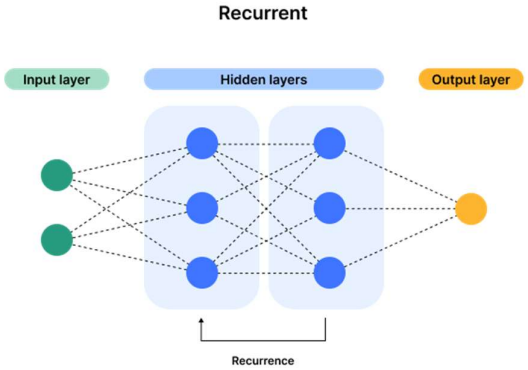
**Multilayer perceptron** neural networks add complexity to perceptron networks, and include a hidden layer.



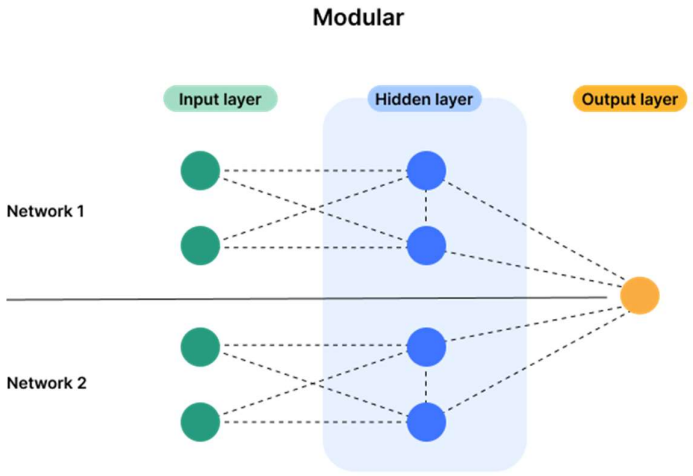

**Feed-forward** neural networks only allow their nodes to pass information to a forward node.



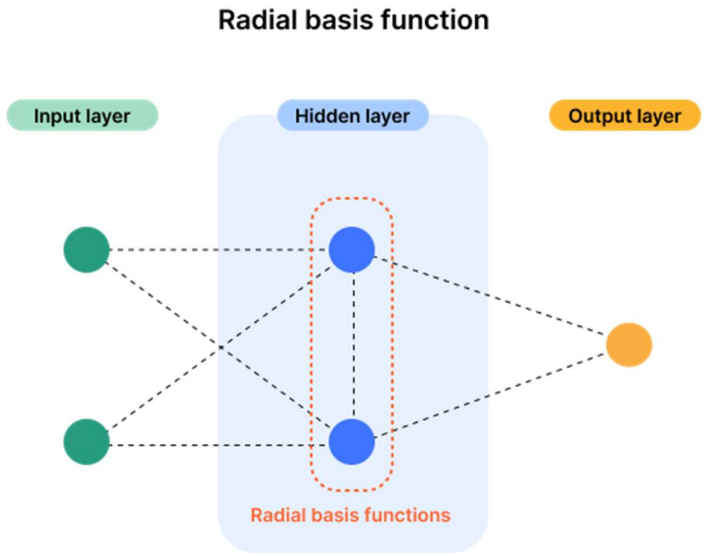

**Recurrent** neural networks can go backwards, allowing the output from some nodes to impact the input of preceding nodes.



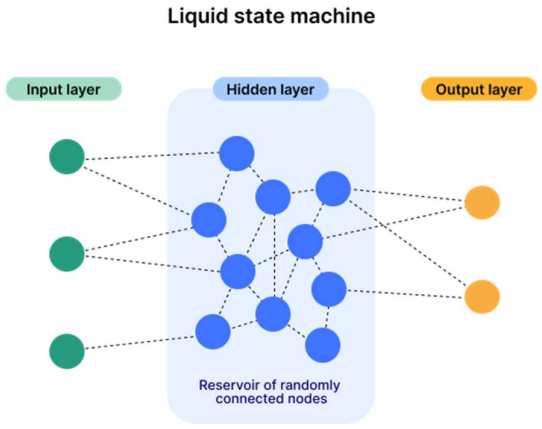

**Modular** neural networks combine two or more neural networks in order to arrive at the output.



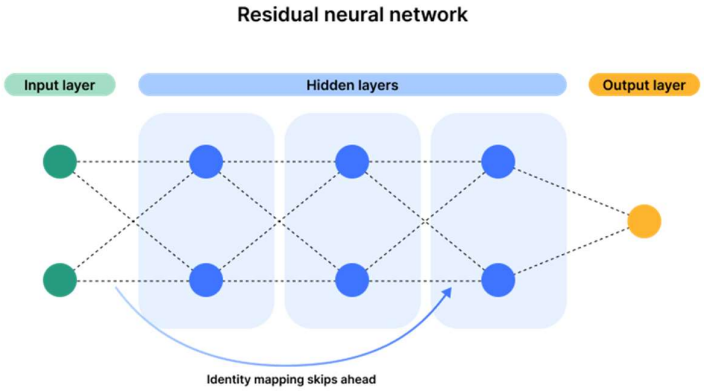

**Radial basis function** neural network nodes use a specific kind of mathematical function called a radial basis function.




**Liquid state machine** neural networks feature nodes that are randomly connected to each other.



**Residual** neural networks allow data to skip ahead via a process called identity mapping, combining the output from early layers with the output of later layers.




**What is a transformer neural network?**

Transformer neural networks are worth highlighting because they have assumed a place of outsized importance in the AI models in widespread use today.

First proposed in 2017, transformer models are neural networks that use a technique called "self-attention" to take into account the context of elements in a sequence, not just the elements themselves. Via self-attention, they can detect even subtle ways that parts of a data set relate to each other.

This ability makes them ideal for analyzing (for example) sentences and paragraphs of text, as opposed to just individual words and phrases. Before transformer models were developed, AI models that processed text would often "forget" the beginning of a sentence by the time they got to the end of it, with the result that they would combine phrases and ideas in ways that did not make sense to human readers. Transformer models, however, can process and generate human language in a much more natural way.

Transformer models are an integral component of generative AI, in particular LLMs that can produce text in response to arbitrary human prompts.

**History of neural networks**

Neural networks are actually quite old. The concept of neural networks can be dated to a 1943 mathematical paper that modeled how the brain could work. Computer scientists began attempting to construct simple neural networks in the 1950s and 1960s, but eventually the concept fell out of favor. In the 1980s the concept was revived, and by the 1990s neural networks were in widespread use in AI research.

However, only with the advent of hyper-fast processing, massive data storage capabilities, and access to computing resources were neural networks able to advance to the point they have reached today, where they can imitate or even exceed human cognitive abilities. Developments are still being made in this field; one of the most important types of neural networks in use today, the transformer, dates to 2017.

**How does Cloudflare support neural networks?**

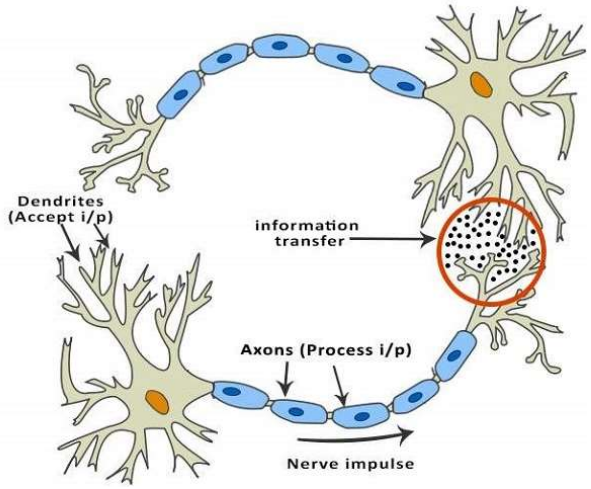
With locations in more than 335 cities around the world, Cloudflare is in a unique position to offer computational power to AI developers anywhere with minimal latency. Cloudflare for AI lets developers run AI tasks on a global network of graphics processing units (GPUs) with no

extra setup. Cloudflare also offers cost-effective cloud storage options for the vast amounts of data required to train neural networks. Learn more about Cloudflare for AI.

Basic Structure of ANNs

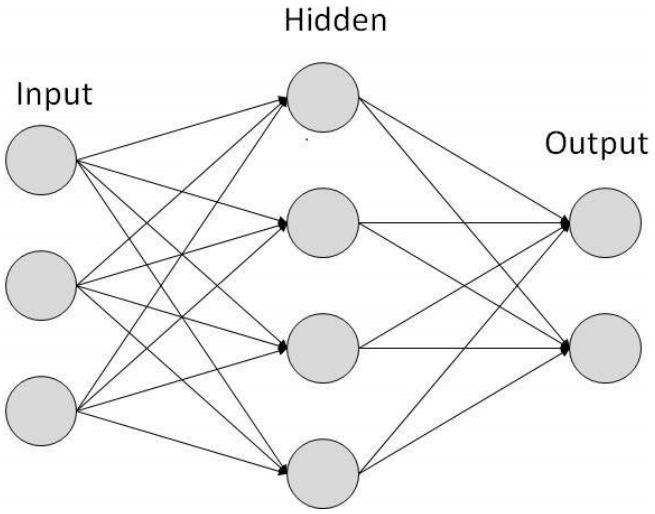
The idea of ANNs is based on the belief that working of human brain by making the right connections, can be imitated using silicon and wires as living **neurons** and **dendrites**.

The human brain is composed of 86 billion nerve cells called **neurons.** They are connected to other thousand cells by **Axons.** Stimuli from external environment or inputs from sensory organs are accepted by dendrites. These inputs create electric impulses, which quickly travel through the neural network. A neuron can then send the message to other neuron to handle the issue or does not send it forward.




ANNs are composed of multiple **nodes**, which imitate biological **neurons** of human brain. The neurons are connected by links and they interact with each other. The nodes can take input data and perform simple operations on the data. The result of these operations is passed to other neurons. The output at each node is called its **activation** or **node value.**

Each link is associated with **weight.** ANNs are capable of learning, which takes place by altering weight values. The following illustration shows a simple ANN −




Artificial Neurons Vs Biological Neurons

Some of the differences between artificial neurons and biological neurons are tabulated below.

| Feature | Biological Neurons | Artificial Neurons |
|---|---|---|
| Structure | Complex with cell body, dendrites, and axons. | Simplifies mathematical function. |
| Functionality | Signals interpreted through action potentials and neurotransmitters. | Computes weighted sums and applies an activation function. |
| Communication | Uses chemical and electrical signals for inter-neuron communication | Communicates through numerical data and mathematical models |
| Processing Speed | Slower transmission due to chemical processes. | Extremely fast computations based on electronic processing. |
| Energy Efficiency | Highly energy efficient and is capable of operating on small amounts of energy. | Generally less energy-efficient, varies by architecture and implementation. |
| Complexity | Highly complex, involves various types of neurons and glacial cells, and intricate chemical signaling. | Simpler in structure but can scale up to model complex behaviors through layers of neurons in neural networks. |
| Execution | Operates in real-time within biological systems. | Operates in digital environments, requiring hardware and software. |
| Learning | Learns through experience and can generalize in complex ways. | Learns through datasets and optimization techniques, often requiring large datasets. |

**Types of ANNs**

Neural networks are classified into different categories based on factors like their depth, the number of hidden layers, and the I/O capabilities of each node. Types of neural networks include −

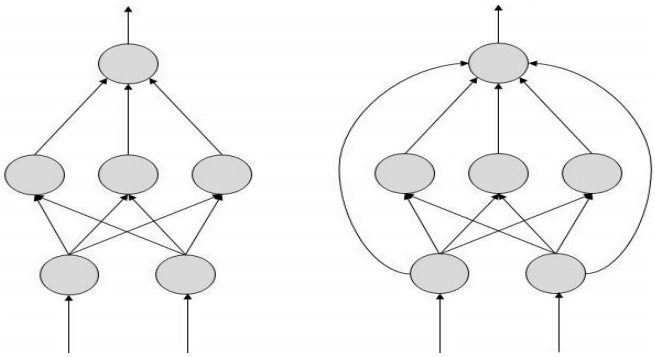
- Convolutional Neural Networks
- Deconvolutional Neural Networks
- Recurrent Neural Networks
- Feed-forward Neural Networks
- Modular Neural Networks
- Generative Adversarial Networks

Topologies of ANNs

There are two Artificial Neural Network topologies − **FeedForward** and **Feedback.**

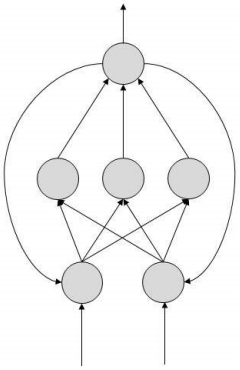
FeedForward ANN

In this ANN, the information flow is unidirectional. A unit sends information to other unit from which it does not receive any information. There are no feedback loops. They are used in pattern generation/recognition/classification. They have fixed inputs and outputs.



**FeedBack ANN**

Feedback ANN have connections that loop back, which allows to fed back information into the structure. This structure enables to handle sequential data and temporal dependencies, making them suitable for tasks like time series prediction and language modeling.



**Working of ANNs**

In the topology diagrams shown, each arrow represents a connection between two neurons and indicates the pathway for the flow of information. Each connection has a weight, an integer number that controls the signal between the two neurons.

If the network generates a â œgood or desiredâ output, there is no need to adjust the weights. However, if the network generates a â œpoor or undesiredâ output or an error, then the system alters the weights in order to improve subsequent results.

Machine Learning in ANNs

ANNs are capable of learning and they need to be trained. There are several learning strategies −

- **Supervised Learning:** It involves a teacher that is scholar than the ANN itself. For example, the teacher feeds some example data about which the teacher already knows the answers.

For example, pattern recognizing. The ANN comes up with guesses while recognizing. Then the teacher provides the ANN with the answers. The network then compares it guesses with the teacherâ ™s â œcorrectâ answers and makes adjustments according to errors.

- **Unsupervised Learning:** It is required when there is no example data set with known answers. For example, searching for a hidden pattern. In this case, clustering i.e. dividing a set of elements into groups according to some unknown pattern is carried out based on the existing data sets present.

- **Reinforcement Learning:** This strategy built on observation. The ANN makes a decision by observing its environment. If the observation is negative, the network adjusts its weights to be able to make a different required decision the next time.

**Applications of ANNs**

They can perform tasks that are easy for a human but difficult for a machine −

- **Aerospace:** Autopilot aircrafts, aircraft fault detection.
- **Automotive:** Automobile guidance systems.
- **Military:** Weapon orientation and steering, target tracking, object discrimination, facial recognition, signal/image identification.
- **Electronics:** Code sequence prediction, IC chip layout, chip failure analysis, machine vision, voice synthesis.
- **Financial:** Real estate appraisal, loan advisor, mortgage screening, corporate bond rating, portfolio trading program, corporate financial analysis, currency value prediction, document readers, credit application evaluators.
- **Industrial:** Manufacturing process control, product design and analysis, quality inspection systems, welding quality analysis, paper quality prediction, chemical product design analysis, dynamic modeling of chemical process systems, machine maintenance analysis, project bidding, planning, and management.
- **Medical:** Cancer cell analysis, EEG and ECG analysis, prosthetic design, transplant time optimizer.
- **Speech:** Speech recognition, speech classification, text to speech conversion.
- **Telecommunications:** Image and data compression, automated information services, real-time spoken language translation.
- **Transportation:** Truck Brake system diagnosis, vehicle scheduling, routing systems.
- **Software:** Pattern Recognition in facial recognition, optical character recognition, etc.
- **Time Series Prediction:** ANNs are used to make predictions on stocks and natural calamities.
- **Signal Processing:** Neural networks can be trained to process an audio signal and filter it appropriately in the hearing aids.
- **Control:** ANNs are often used to make steering decisions of physical vehicles.
- **Anomaly Detection:** As ANNs are expert at recognizing patterns, they can also be trained to generate an output when something unusual occurs that misfits the pattern.

In AI, emergent systems refer to complex behaviors and capabilities that arise from the interactions of simpler elements, exceeding what's explicitly programmed, and are often unexpected.

Here's a more detailed explanation:

- **Definition:**

Emergent behavior in AI occurs when simple components, like neurons in a neural network or agents in a multi-agent system, interact in non-linear ways, leading to complex behaviors that are not explicitly programmed.

- **Examples:**
  - **Large Language Models (LLMs):** LLMs, when trained on vast amounts of data, can exhibit abilities beyond their intended purpose, such as understanding context better than expected or generating creative content.

    - **Autonomous Vehicles:** Self-driving cars, leveraging a blend of sensors, software, and AI algorithms, learn from their environment and navigate complex scenarios unpredicted by their initial programming.
- **Significance:**
    - Emergent AI represents a shift towards more autonomous, self-improving AI systems that could unlock great potential across industries.
    - These systems can anticipate needs without explicit mention, offer deep insights, and collaborate in ways that were unimaginable even a few years ago.
- **Potential Risks:**
    - Emergent AI also raises concerns about unintended consequences and the need for careful design and oversight.
    - For example, algorithms may not be adjusted to consider new forms of knowledge, leading to emergent bias.